ARTIFACT EVALUATED usenix ASSOCIATION AVAILABLE
ARTIFACT EVALUATED usenix ASSOCIATION FUNCTIONAL
ARTIFACT EVALUATED usenix ASSOCIATION REPRODUCED

# DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving

Yinmin Zhong[1] Shengyu Liu[1] Junda Chen[3] Jianbo Hu[1] Yibo Zhu[2] Xuanzhe Liu[1]
Xin Jin[1] Hao Zhang[3]

[1]*School of Computer Science, Peking University* [2]*StepFun* [3]*UC San Diego*

arXiv:2401.09670v3 [cs.DC] 6 Jun 2024

## Abstract

DistServe improves the performance of large language models (LLMs) serving by disaggregating the prefill and decoding computation. Existing LLM serving systems colocate the two phases and batch the computation of prefill and decoding across all users and requests. We find that this strategy not only leads to strong prefill-decoding interferences but also couples the resource allocation and parallelism plans for both phases. LLM applications often emphasize individual latency for each phase: time to first token (TTFT) for the prefill phase and time per output token (TPOT) of each request for the decoding phase. In the presence of stringent latency requirements, existing systems have to prioritize one latency over the other, or over-provision compute resources to meet both.

DistServe assigns prefill and decoding computation to different GPUs, hence eliminating prefill-decoding interferences. Given the application's TTFT and TPOT requirements, DistServe co-optimizes the resource allocation and parallelism strategy *tailored* for each phase. DistServe also places the two phases according to the serving cluster's bandwidth to minimize the communication caused by disaggregation. As a result, DistServe significantly improves LLM serving performance in terms of the maximum rate that can be served within both TTFT and TPOT constraints on each GPU. Our evaluations show that on various popular LLMs, applications, and latency requirements, DistServe can serve 7.4× more requests or 12.6× tighter SLO, compared to state-of-the-art systems, while staying within latency constraints for $> 90\%$ of requests.

## 1 Introduction

Large language models (LLMs), such as GPT-4 [37], Bard [2], and LLaMA [51], represent a groundbreaking shift in generative AI. They start to reshape existing Internet services, ranging from search engines to personal assistants [4], and enable fundamentally new applications, like universal chatbots [1, 16] and programming assistants [15, 42]. Yet, these advances come with a significant challenge: processing an end-to-end LLM query can be substantially slower than a standard search query [41]. In order to meet the stringent latency requirements of various applications, service providers need to over-provision compute resources, particularly many GPUs, leading to a shortfall in cost efficiency. Therefore, optimizing the cost per LLM query while adhering to high *SLO attainment* (the proportion of requests that meet the SLOs) is becoming increasingly essential for all LLM services.

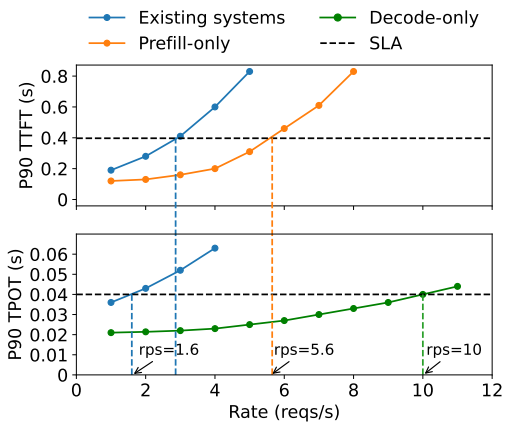



Figure 1: Performance when serving an LLM with 13B parameters under a synthetic workload with input length = 512 and output length = 64 on one NVIDIA 80GB A100. *Upper*: The P90 time-to-first-token (TTFT) latency comparing existing systems vs. a system serving only the prefill phase. *Down*: The P90 time-per-output-token (TPOT) latency comparing existing systems vs. a system serving only the decoding phase.

An LLM service responds to a user query in two phases. The *prefill phase* processes a user's prompt, composed of a sequence of tokens, to generate the first token of the response *in one step*. Following it, the *decoding phase* sequentially generates subsequent tokens *in multiple steps*; each decoding step generates a new token based on tokens generated in previous steps, until reaching a termination token. This dual-phase process distinguishes LLM services from traditional services – an LLM service's latency is uniquely measured by two key metrics: the *time to first token* (TTFT), which is the duration of the prefill phase, and the *time per output*

*token* (TPOT), which represents the average time taken to generate a token for each request (except for the first token)[1]. Different applications place varying demands on each metric. For example, real-time chatbots [1] prioritize low TTFT for response promptness, while TPOT only remains important until it is faster than human reading speed (i.e., 250 words/min). Conversely, document summarization emphasizes low TPOT for faster generation of the summary.

Hence, given the application's TTFT and TPOT requirements, an effective LLM serving system should balance these needs and maximize *per-GPU goodput*, defined as the maximum request rate that can be served adhering to the SLO attainment goal (say, 90%) for each GPU provisioned – higher per-GPU goodput directly translates into lower cost per query.

As the prefill and decoding phases share the LLM weights and working memory, existing LLM serving systems typically colocate both phases on GPUs and maximize the overall system throughput – tokens generated per second across all users and requests – by batching the prefill and decoding steps across requests [31, 54]. However, to meet latency requirements, we find these systems must over-provision compute resources. To see this, Figure 1 illustrates how the P90 TTFT and TPOT shift with increasing request rates when serving a 13B LLM using existing systems [32], with workload pattern and two latency constraints set to emulate using LLM to generate a short summary for an article. Under the SLO attainment of 90%, the maximum achievable goodput on a single A100 GPU, which is constrained by the more stringent one of TTFT and TPOT requirements, is about 1.6 requests per second (rps). The performance contrasts sharply when each phase is served independently on a separate GPU, shown by the orange and green curves, which achieve per-GPU goodput of 5.6 rps for the prefill phase and 10 rps for decoding. Ideally, by allocating 2 GPUs for prefill and 1 GPU for decoding, we can effectively serve the model with an overall goodput of 10 rps, or equally 3.3 rps per GPU, which is 2.1x higher than existing systems. The gap in goodput primarily stems from the colocation of the prefill and decoding – two phases with very distinct computational characteristics and latency requirements (§2.1).

First, colocation leads to strong *prefill-decoding interference*. A prefill step often takes much longer than a decoding step. When batched together, decoding steps in the batch are delayed by the prefill steps, significantly elongating their TPOT; similarly, the inclusion of decoding steps contributes to a non-trivial increase in TTFT, as evidenced in Figure 2. Even if we schedule them separately, issues persist as they begin to compete for resources. Decoding tasks awaiting GPU execution are subject to increased queuing delays due to ongoing prefill tasks, and vice versa. Prioritized scheduling of one phase risks failing the latency requirements of the other.

Second, the prefill and decoding computation differ in latency requirements and preference for different forms of parallelism (§3). Colocating prefill and decoding, however, couples their resource allocation, and prevents implementing different parallelism strategies more suited to meeting the specific latency requirements of each phase.

To overcome these challenges, we propose to disaggregate the prefill and decoding phases of LLM inference, assigning them to separate GPUs. Our approach has two benefits. First, operating each phase independently on different GPUs eliminates prefill-decoding interference. Second, it allows to scale each phase independently with tailored resource allocation and model parallelism strategies to meet their specific latency requirements. Although disaggregation causes communication of intermediate states between GPUs, we show that the communication overhead is insubstantial (§3.3) in modern GPU clusters, and when managed appropriately, disaggregation significantly improves per-GPU goodput.

Based on the above insights, in this work, we build DistServe[2], a goodput-optimized LLM serving system by disaggregating the prefill and decoding phases. Given TTFT and TPOT requirements, DistServe first scales each phase independently by co-optimizing the GPU allocation and parallelism strategies of the prefill and decoding phase assuming serving a single model replica. The optimization ensures maximizing the per-GPU goodput and may assign different numbers of GPUs and parallelism strategies to each phase depending on their respective latency requirements. DistServe then scales this allocation to multiple instances via replication until meeting the user-required traffic rate (§4). DistServe also features an algorithm to place the prefill and decoding computation according to their allocation schemes and the cluster's bandwidth to minimize the overhead of communicating intermediate states between phases.

We implement DistServe as an orchestration layer on top of the LLM inference engine. We evaluate DistServe on various LLMs, varying the workloads based on three important real-world LLM applications: chatbots, programming assistant, and document summary. Compared to state-of-the-art solutions, DistServe can serve up to 7.4× more requests or 12.6× tighter SLO under various latency constraints. Our contributions are:

- Identify the problems of prefill-decoding interference and resource coupling in existing LLM serving systems and propose to disaggregate the two phases.
- Design a novel placement algorithm to choose the goodput-optimal schema for prefill and decoding instances automatically.
- Conduct a comprehensive evaluation of DistServe with realistic workloads.

## 2 Background and Motivation

An LLM service follows a client-server architecture: the client submits a sequence of text as a request to the server; the server

[1]The overall request latency equals TTFT plus TPOT times the number of generated tokens in the decoding phase.

[2]https://github.com/LLMServe/DistServe

hosts the LLM on GPUs, runs inference over the request, and responds (or streams) the generation back to the client. As explained in §1, due to the unique prefill-decoding process, LLM service may impose aggressive service-level objectives (SLOs) on both TTFT and TPOT, varying with the application's needs. The serving system must meet both SLOs while minimizing the cost associated with expensive GPUs. In other words, we want the serving system to maximize the requests served per second adhering to the SLO attainment goal for each GPU provisioned – *maximizing per-GPU goodput*. Next, we detail the LLM inference computation (§2.1) and discuss existing optimizations for LLM serving (§2.2).

## 2.1 LLM Inference

Modern LLMs [37, 51] predict the next token given an input sequence. This prediction involves computing a hidden representation for each token within the sequence. An LLM can take a variable number of input tokens and compute their hidden representations in parallel, and its computation workload increases superlinearly with the number of tokens processed in parallel. Regardless of the input token count, the computation demands substantial I/O to move LLM weights and intermediate states from the GPU's HBM to SRAM. This process is consistent across varying input sizes.

The prefill step deals with a new sequence, often comprising many tokens, and processes these tokens concurrently. Unlike prefill, each decoding step only processes one new token generated by the previous step. This leads to significant computational differences between the two phases. When dealing with user prompts that are not brief, the prefill step tends to be compute-bound. For instance, for a 13B LLM, computing the prefill of a 512-token sequence makes an A100 near compute-bound (see §3.1). In contrast, despite processing only one new token per step, the decoding phase incurs a similar level of I/O to the prefill phase, making it constrained by the GPU's memory bandwidth.

During both phases, intermediate states, known as KV caches [32], are generated at each token position, which are needed again in later decoding steps. To avoid recomputing them, they are saved in GPU memory. Because of the shared use of LLM weights and KV caches in memory, most LLM inference engines opt to colocate the prefill and decoding phases on GPUs, despite their distinct computational characteristics.

## 2.2 LLM Serving Optimization

In real-time online serving, multiple requests come and must be served within SLOs. Batching and parallelizing their computation is key for achieving low latency, high throughput, and high utilization of GPUs.

**Batching.** Current serving systems [9, 32, 54] utilize a batching technique known as *continuous batching*. This method batches the prefill of new requests with the decoding of ongoing ones. It boosts the GPU utilization and maximizes the overall system throughput – tokens generated per second across all users and requests. However, as mentioned in §1 and elaborated later in §2.3, this approach leads to trade-offs between TTFT and TPOT. An advanced variant of continuous batching [9] attempts to balance TTFT and TPOT by segmenting long prefill into chunks and attaching decoding jobs with a chunked prefill – but essentially, it trades TTFT for TPOT and cannot eliminate the interference (§2.3). In summary, batching prefill and decoding invariably leads to compromises in either TTFT or TPOT.

**Model parallelism.** In LLM serving, model parallelism is generally divided as intra- and inter-operator parallelisms [33, 46, 59]. Both can be used to support larger models but may impact serving performance differently. Intra-operator parallelism partitions computationally intensive operators, such as matrix multiplications, across multiple GPUs, accelerating computation but causing substantial communication. It reduces the execution time[3], hence latency, particularly for TTFT of the prefill phase, but requires high bandwidth connectivity between GPUs (e.g., NVLINK). Inter-operator parallelism organizes LLM layers into stages, each running on a GPU to form pipelines. It moderately increases execution time due to inter-stage communication, but linearly scales the system's rate capacity with each added GPU. In this paper, we reveal an additional benefit of model parallelism: reduced queuing delay of both prefill and decoding phases, steaming from shorter execution time. We delve into this further in §3. Besides model parallelism, replicating a model instance, irrespective of its model parallelism configurations, linearly scales the system's rate capacity.

These parallelism strategies create a complex space of optimization that requires careful trade-offs based on the application's latency requirements.

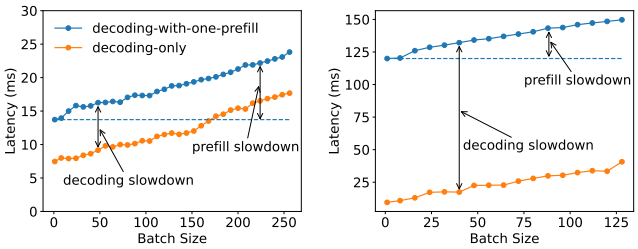
## 2.3 Problems and Opportunities

Colocating and batching the prefill and decoding computation to maximize the overall system throughput, as in existing systems, is cost-effective for service providers. However, in the presence of SLOs, present approaches struggle to maintain both high service quality and low cost due to the issues discussed below.

**Prefill-decoding interference.** As Figure 2 shows, adding a single prefill job to a batch of decoding requests significantly slows down both processes, leading to a marked increase in TTFT and TPOT. Specifically, the decoding tasks in the batch must wait for lengthier prefill jobs to complete, thus extending TPOT; the slowdown intensifies with a longer prefill, shown in Figure 2(b). Adding decoding jobs to prefill also increases the time to complete the prefill task, particularly when the GPU is already at capacity (Figure 2 blue curves).

One attempt to mitigate this interference is called *chunked-prefill with piggyback* [3, 9]. It proposes to split the long prefill

[3] we emphasize "execution time" instead of latency here because latency comprises both execution time and queuing delay.




(a) Input length = 128 (b) Input length = 1024

Figure 2: Batch execution time when serving a 13B LLM as batch size increases. Compared between a decoding-only batch and the batch adding one more prefill job.

into chunks and batch a prefill chunk with a few decoding jobs (a.k.a. piggybacking). This technique alleviates the slowdown of the decoding job caused by the long prefill job, but it does not eliminate it. Additionally, it results in an extra overhead for the prefill job which cannot be easily mitigated by adjusting the chunk size. First, if the chunk size is set much lower than the inflection point that can saturate the GPU, then the prefill job will have a longer execution time since it competes with the decoding job in the same batch and cannot solely utilize the GPU resources. Second, if we increase the chunk size to nearly saturate the GPU, the chance of piggybacking will diminish since the remaining slots for decode tokens are limited. Also, chunked-prefill causes significantly more memory access for the prefill jobs. This is because the KV cache of all previous chunks have to be loaded from HBM to SRAM repeatedly to compute each subsequent chunk. Concretely, if a prefill job is split into $N$ equal chunks, we need to load $N+(N-1)+...+1 = O(N^2)$ chunks of KV Cache in total, compared to O(N) in the non-chunked case. This overhead will increase as the context length becomes longer.

**Ineffective scheduling.** Unbatching prefill and decoding jobs and scheduling them sequentially does not mitigate the interference. Decoding jobs may experience longer queuing delays due to waiting for ongoing prefill jobs on GPUs. Moreover, batches dedicated to decoding often lead to GPU underutilization. Prioritizing tasks in either phase adversely affects the latency of the other, rendering priority scheduling ineffective.

**Resource and parallelism coupling.** Colocating prefill and decoding phases on the same GPUs unavoidably share their resource and parallelism settings. However, each phase has its unique computational characteristic and latency requirement that calls for more heterogeneous resource allocation. For example, the prefill phase tends to be compute-bound and benefits from more intra-op parallelism to reduce execution time to meet the tight SLO on TTFT. By contrast, the optimal parallelism configuration of the decoding phase depends on the running batch size. In existing systems, due to coupling, resource allocation and parallelism plans are tailored to satisfy the *more demanding* of TTFT and TPOT, which may not be ideal for the other. This often leads to resource over-provisioning to meet both SLOs.

**Opportunities.** To address these issues, we propose to disaggregate the prefill and decoding phases. We use the term *instance* to denote a unit of resources that manages exactly one complete copy of model weights. One instance can correspond to many GPUs when model parallelism is applied. Note that when we disaggregate the two phases to different GPUs, each phase manages its copy of the model weights, resulting in *prefill instances* and *decoding instances*. A prefill instance, upon receiving a request, performs only the prefill computation for this request to generate the first output token. It then sends the intermediate results (mainly KV caches) to a decoding instance, which is responsible for subsequent decoding steps. Because decoding computation often has low GPU utilization, we may allocate multiple prefill instances per decoding instance. This allows batching more decoding jobs to achieve higher GPU utilization.

Disaggregating prefill and decoding naturally resolves the interference between the two phases and enables each to focus on its optimization target – TTFT or TPOT. Each type of instance can employ different resources and parallelism strategies to meet a variety of latency requirements. By adjusting the number of GPUs and parallelisms provided to the two types of instances, we can maximize the per-device goodput of the overall system, avoiding over-provisioning, eventually translating to reduced cost-per-query adhering to service quality. Next, we develop ways to find out the best resource allocation and parallelism plan for each phase.

## 3 Tradeoff Analysis

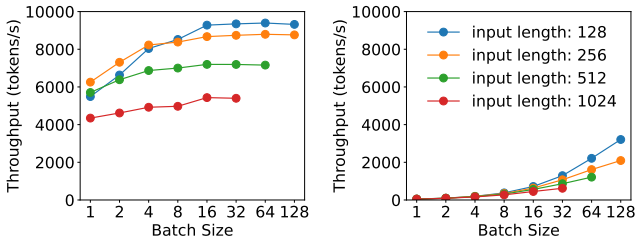
Disaggregation uncouples the two phases and allows a distinct analysis of the characteristics of each phase, providing valuable insights into the algorithm design. It also expands the design space: now each phase needs to be scaled and scheduled independently based on their latency requirements.

In this section, we analyze the computational pattern of prefill (§3.1) and decoding instances (§3.2) *post disaggregation*. We aim to identify key parameters and derive guidelines for batching and parallelism in each phase. We then highlight several practical deployment considerations (§3.3). This section lays the foundation for per-gpu goodput optimization.

### 3.1 Analysis for Prefill Instance

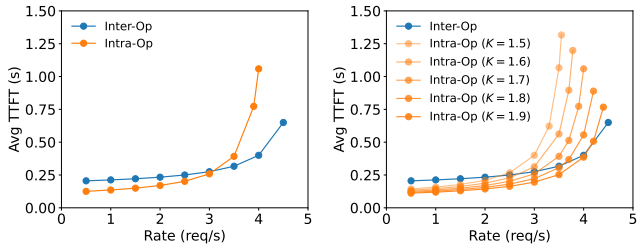
After disaggregation, the prefill phase generates the first token by processing all tokens of the user prompt in parallel. Assuming a given arrival rate, we aim to fulfill the service's latency requirement on TTFT using the least resources.

**Batching strategy.** The prefill step is typically compute-intensive. Figure 3(a) shows how the throughput of the prefill phase changes with the input length and the batch size. For a 13B parameter LLM, processing a single sequence of 512 tokens can fully engage an A100 GPU. Once the GPU becomes compute-bound, adding more requests to the batch no longer improves GPU efficiency. Instead, it proportionally extends the total processing time for the batch, inadvertently




(a) Prefill phase (b) Decoding phase

Figure 3: Throughput for two phases with different batch sizes and input lengths when serving an LLM with 13B parameters.




(a) Real experiment results (b) Changing intra-op speedup

Figure 4: Average TTFT when serving an LLM with 66B parameters using different parallelism on two A100 GPUs.

delaying all included requests. Hence, for prefill instances, it is necessary to profile the specific LLM and GPUs in advance to identify a critical input length threshold, denoted as $L_m$, beyond which the prefill phase becomes compute-bound. Batching more requests should only be considered when the input length of the scheduled request is below $L_m$. In practice, user prompts typically average over hundreds of tokens [8]. Batch sizes for the prefill instance are generally kept small.

**Parallelism plan.** To study the parallelism preferences for prefill-only instances, we serve a 66B LLM on two A100 GPUs with inter-op or intra-op parallelism strategy. To simplify the problem, we assume uniform requests input lengths of 512 tokens and a Poisson arrival process. We compare the resulting average TTFT at various arrival rates in Figure 4(a): intra-op parallelism is more efficient at lower arrival rates, while inter-op parallelism gains superiority as the rate increases. Disaggregation enables the prefill phase to function analogously to an M/D/1 queue, so we can use queuing theory to verify the observation.

We start by developing notations using the single-device case without parallelism: each request's execution time, denoted as $D$, remains constant due to uniform prefill length. Since one request saturates the GPU, we schedule requests via First-Come-First-Served (FCFS) without batching. Suppose the Poisson arrival rate is $R$ and the utilization condition of $RD < 1$, the average TTFT ($Avg_TTFT$) can be modeled by the M/D/1 queue [47] in close form:

$$Avg_TTFT = D + \frac{RD^2}{2(1-RD)}. \tag{1}$$

where the first term represents the execution time and the second corresponds to the queuing delay. Based on Eq. 1, we incorporate parallelism below.

With 2-way inter-op parallelism, we assume the request-level latency becomes $D_s$, and the slowest stage takes $D_m$ to finish. We have $D \approx D_s \approx 2 \times D_m$, due to negligible inter-layer activation communication [33, 59]. The average TTFT with 2-way inter-op parallelism is derived as:

$$Avg_TTFT_{inter} = D_s + \frac{RD_m^2}{2(1-RD_m)} = D + \frac{RD^2}{4(2-RD)}. \tag{2}$$

For intra-op parallelism, we introduce a speedup coefficient $K$, where $1 < K < 2$, reflecting the imperfect speedup caused by high communication overheads of intra-op parallelism. With the execution time $D_s = \frac{D}{K}$, the average TTFT for 2-degree intra-op parallelism is:

$$Avg_TTFT_{intra} = \frac{D}{K} + \frac{RD^2}{2K(K-RD)}. \tag{3}$$

Comparing Eq. 2 and Eq. 3: at lower rates, where execution time (first term) is the primary factor, intra-op parallelism's reduction in execution time makes it more efficient. As the rate increases and the queuing delay (second term) becomes more significant, inter-op parallelism becomes advantageous, concurred with Figure 4(a).

The prefill phase's preference for parallelism is also influenced by TTFT SLO and the speedup coefficient $K$. Seen from Figure 4(a): A more stringent SLO will make intra-op parallelism more advantageous, due to its ability to reduce execution time. The value of K depends on factors such as the input length, model architecture, communication bandwidth, and placement [46,59]. As shown in Figure 4(b), a decrease in K notably reduces the efficacy of intra-op parallelism. §4 develops algorithms that optimize the resource and parallelism configurations taking into consideration these knobs.

## 3.2 Analysis for Decoding Instance

Unlike the prefill instance, a decoding instance follows a distinct computational pattern: it receives the KV caches and the first output token from the prefill instance and generates subsequent tokens one at a time. For decoding instances, our optimization goal is to satisfy the application's TPOT requirement using minimal computing resources.

**Batching strategy.** Since a single decoding job is heavily bandwidth-bound, batching is key to avoiding low GPU utilization (hence high per-gpu goodput), as shown in Figure 3(b). In existing systems where the prefill and decoding phases are colocated, increasing the decoding batch size is difficult because it conflicts with meeting latency goals, particularly in scenarios with high request rates. This is because sharing GPUs cause competition between prefill and decoding jobs, leading to a trade-off between TTFT and TPOT. For

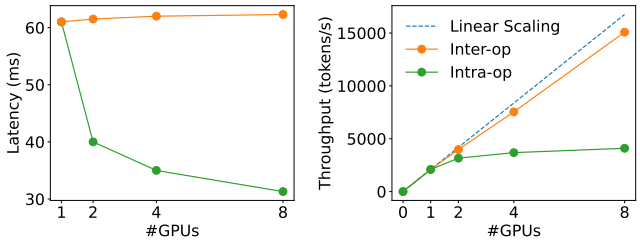



Figure 5: Decoding phase latency and throughput when serving a 13B LLM with batch size = 128 and input length = 256 under different parallel degrees.

example, a higher arrival rate generates more prefill jobs, demanding greater GPU time to meet TTFT requirements if prioritizing prefill jobs, which in turn adversely affects TPOT.

On the contrary, disaggregation offers a solution by enabling the allocation of multiple prefill instances to a single decoding instance. This approach allows for accumulating a larger batch size on dedicated GPUs for the decoding phase without sacrificing TPOT.

**Parallelism plan.** Post-disaggregation, the batch size for decoding may be constrained by GPU memory capacity, as it is necessary to maintain the KV caches for all active requests. Scaling the decoding instance with model parallelism or leveraging advanced memory management techniques for LLM KV caches, such as Paged-Attention [32] and GQA [10], enable further scaling of the decoding batch size to nearly compute-bound. As the decoding batch size continue to increase to approach the compute-bound, the decoding computation begins to resemble the prefill phase. With this observation, we investigate how the latency and throughput change under different parallelism degrees under large batch conditions in Figure 5: intra-op parallelism reduces latency with diminishing returns, caused by communication and reduced utilization after partitioning. Inter-op parallelism can almost linearly scale the throughput. Hence, when the TPOT SLO is stringent, intra-op parallelism is essential to reduce TPOT to meet latency goals. Beyond this, inter-op parallelism is preferable to enhance throughput linearly.

It is worth noting that when the model can fit into the memory of a single GPU, replication is a competitive option in addition to model parallelism for both prefill and decoding instances, to linearly scale the system's rate capacity. It may also reduce the queuing delay – as indicated by Eq. 1 – by substituting $R$ with $R/N$ assuming requests are equally dispatched to $N$ replicas, at the cost of maintaining additional replicas of the model weights in GPU memory.

## 3.3 Practical Problems

We have developed foundational principles for selecting batching and parallelisms for each phase. In this section, we discuss and address several challenges encountered during the practical deployment of disaggregated prefill and decoding phases.

**Variable prefill length.** §3 has assumed uniform prompt length across requests. In real deployments, depending on the LLM application, the lengths of requests are non-uniform. The non-uniformity can cause pipeline bubbles [28, 36] for prefill instances applying inter-op parallelism because the execution time of pipeline stages across requests of different lengths will vary. This results in slight deviations from the conclusions indicated by using the M/D/1 queue model. To address the problem, §4 develops algorithms that search for parallelisms based on workloads, and resort to scheduling to minimize the bubbles (§4.3).

**Communication overhead.** Transferring KV caches from prefill to decoding instances incurs notable overheads. For example, the KV cache size of a single 512-token request on OPT-66B is approximately 1.13GB. Assuming an average arrival rate of 10 rps, we need to transfer 11.3GB data per second—or equivalently 90Gbps bandwidth to render the overhead invisible. While many modern GPU clusters for LLMs are equipped with InfiniBand (e.g., 800 Gbps), in cases where cross-node bandwidth is limited, DistServe relies on the commonly available intra-node NVLINK, where the peak bandwidth between A100 GPUs is 600 GB/s, again rendering the transmission overhead negligible (see §6.3). However, this requirement imposes additional constraints on the placement of prefill and decoding instances that we take into consideration in the next section.

Through the analysis in this section, we identify the workload pattern, placement constraints, SLO requirements, parallelism strategies, and resource allocation as key parameters that create a web of considerations in designing the disaggregated serving system. How to automatically navigate the search space to find the configuration that achieves optimal per-gpu goodput is challenging, and addressed next.

# 4 Method

We built DistServe to solve the above challenges. Given the model, workload characteristic, latency requirements, and SLO attainment target, DistServe will determine (a) the parallelism strategies for prefill and decoding instances, (b) the number of each instance type to deploy, as well as (c) how to place them onto the physical cluster. We call the solution a *placement*. Our goal is to find a placement that maximizes the per-gpu goodput.

As explained in §3.3, a key design consideration is to manage communications between disaggregated prefill and decoding phases, given varying cluster setups. In this section, we first present two placement algorithms: one for clusters with high-speed cross-node networks (§4.1) and the other for environments lacking such infrastructure (§4.2); the latter introduces additional constraints. We then develop online scheduling optimizations that adapt to the nuances of real-world workloads (§4.3).

**Algorithm 1** High Node-Affinity Placement Algorithm

**Input:** LLM $G$, #node limit per-instance $N$, #GPU per-node $M$, GPU memory capacity $C$, workload $W$, traffic rate $R$.
**Output:** the placement *best_plm*.

$config_p, config_d \leftarrow \emptyset, \emptyset$
**for** $intra_op \in \{1, 2, ..., M\}$ **do**
  **for** $inter_op \in \{1, 2, ..., \frac{N \times M}{intra_op}\}$ **do**
    **if** $\frac{G.size}{inter_op \times intra_op} < C$ **then**
      $config \leftarrow (inter_op, intra_op)$
      $\hat{G} \leftarrow \text{parallel}(G, config)$
      $config.goodput \leftarrow \text{simu_prefill}(\hat{G}, W)$
      **if** $\frac{config_p.goodput}{config_p.num_gpus} < \frac{config.goodput}{config.num_gpus}$ **then**
        $config_p \leftarrow config$
      $config.goodput \leftarrow \text{simu_decode}(\hat{G}, W)$
      **if** $\frac{config_d.goodput}{config_d.num_gpus} < \frac{config.goodput}{config.num_gpus}$ **then**
        $config_d \leftarrow config$
$n, m \leftarrow \lceil \frac{R}{config_p.goodput} \rceil, \lceil \frac{R}{config_d.goodput} \rceil$
$best_plm \leftarrow (n, config_p, m, config_d)$
**return** *best_plm*

**Algorithm 2** Low Node-Affinity Placement Algorithm

**Input:** LLM $G$, #node limit per-instance $N$, #GPU per-node $M$, GPU memory capacity $C$, workload $W$, traffic rate $R$.
**Output:** the placement *best_plm*.

$config^* \leftarrow \emptyset$
**for** $inter_op \in \{1, 2, ..., N\}$ **do**
  $\mathcal{P} \leftarrow \text{get_intra_node_configs}(G, M, C, inter_op)$
  **for** $P_p \in \mathcal{P}$ **do**
    **for** $P_d \in \mathcal{P}$ **do**
      **if** $P_p.num_gpus + P_d.num_gpus \leq M$ **then**
        $config \leftarrow (inter_op, P_p, P_d)$
        $\hat{G}_p, \hat{G}_d \leftarrow \text{parallel}(G, config)$
        $config.goodput \leftarrow \text{simulate}(\hat{G}_p, \hat{G}_d, W)$
        **if** $\frac{config.^*goodput}{config.^*num_gpus} < \frac{config.goodput}{config.num_gpus}$ **then**
          $config^* \leftarrow config$
$n \leftarrow \lceil \frac{R}{config.^*goodput} \rceil$
$best_plm \leftarrow (n, config^*)$
**return** *best_plm*

## 4.1 Placement for High Node-Affinity Cluster

On high node-affinity clusters equipped with Infiniband, KV caches transmission overhead across nodes is negligible, DistServe can deploy prefill and decoding instances across any two nodes without constraints. We propose a two-level placement algorithm for such scenarios: we first optimize the parallelism configurations for prefill and decoding instances separately to attain phase-level optimal per-gpu goodput; then, we use replication to match the overall traffic rate.

However, finding the optimal parallel configuration for a single instance type, such as for the prefill instance, is still challenging, due to the lack of a simple analytical formula to calculate the SLO attainment (a.k.a., percentage of requests that meet TTFT requirement), given that the workload has diverse input, output lengths, and irregular arrival patterns. Gauging the SLO via real-testbed profiling is time-prohibitive. We thus resort to building a simulator to estimate the SLO attainment, assuming prior knowledge of the workload's arrival process and input and output length distributions. Although short-term interval is impossible to predict, the workload pattern over longer timescales (e.g., hours or days) is often predictable [33, 55]. DistServe fits a distribution from the history request traces and resamples new traces from the distribution as the input workload to the simulator to compute the SLO attainment. Next, DistServe simply enumerates the placements and finds the maximum rate that meets the SLO attainment target with binary search and simulation trials.

Algorithm 1 outlines the process. We enumerate all feasible parallel configurations, subject to cluster capacity limit, for both prefill and decoding instances. Then, for a specific prefill phase configuration, we use `simu_prefill` to simulate and find its maximum goodput via binary search (similarly for using `simu_decode` for decoding). After determining the optimal parallel configurations for both prefill and decoding instances, we replicate them to achieve the user-required overall traffic rate according to their goodput.

The complexity of Algorithm 1 is $O(NM^2)$, with $N$ as the node limit per instance and $M$ representing the typical number of GPUs per node in modern clusters (e.g., 8). The search space is manageable and the solving time is under 1.3 minutes in our largest setting, as demonstrated in §6.5.

**Simulator building.** Algorithm 1 relies on a simulator to estimate the goodput under various SLOs and SLO attainment goals given the workload and the parallelism plan. To build an accurate simulator, we analyze the FLOPs and the number of memory accesses for prefill and decoding phases respectively, and use a latency model to approximate the inference execution time. See details in Appendix A. The simulator aligns well with real profiling results, thanks to the high predictability of DNN workloads [23, 33], verified in §6.4.

By far, we have developed Algorithm 1 assuming we can place the prefill and decoding instance between any two nodes (or on the same node) of the cluster, and the KV cache transmission utilizes high bandwidth network. In many real clusters, GPUs inside a node access to high-bandwidth NVLINK while GPUs distributed across nodes have limited bandwidth. We next develop an algorithm to address this constraint.

## 4.2 Placement for Low Node-Affinity Cluster

A straightforward solution is to always colocate prefill and decoding instances on the same node, utilizing the NVLINK, which is commonly available inside a GPU node. For large models, e.g. with 175B parameters (350GB), we may be unable to even host a single pair of prefill and decoding instances

in an 8-GPU node ($80G \times 8 = 640G < 350 \times 2GB$). We incorporate this as additional placement constraints and co-optimize it with model parallelism, presented in Algorithm 2.

The key insight is that KV cache transfer occurs exclusively between corresponding layers of prefill and decoding instances. Leveraging inter-op parallelism, we group layers into stages and divide each instance into segments, termed as *instance segments*, with each segment maintaining one specific inter-op stage. By colocating prefill and decoding segments of the same stage within a single node, we force the transfer of intermediate states to occur only via NVLINK. Inside a node, we set the same parallelism and resource allocation for segments of the same instance. Given the typical limitation of GPUs per node (usually 8), we can enumerate possible configurations inside one node and use the simulator to identify the configurations that yield the best goodput.

As outlined in Algorithm 2, we begin by enumerating inter-op parallelism degrees to get all the possible instance segments. For each segment, we get all possible intra-node parallelism configurations by calling `get_intra_node_configs`. Then we use simulation to find the optimal one and replicate it to satisfy the target traffic rate.

## 4.3 Online scheduling

The runtime architecture of DistServe is shown in Figure 6. DistServe operates with a simple FCFS scheduling policy. All incoming requests arrive at a centralized controller, then dispatched to the prefill instance with the shortest queue for prefill processing, followed by dispatch to the least loaded decoding instance for decoding steps. This setup, while simple, is optimized with several key enhancements tailored to the nuances of real-world workloads.

**Reducing pipeline bubbles.** To mitigate the pipeline bubbles caused by non-uniform prompt lengths (§3.3), we schedule the requests in a way that balances the execution time across all batches in the pipeline. This is achieved by noting that, for both prefill and decoding instances, the number of new tokens in the batch is a reliable indicator of the batch's real execution time. For prefill instances, we profile the target model and GPU to figure out the shortest prompt length $L_m$ needed to saturate the GPU. We schedule prefill batches with a total sequence length close to $L_m$, by either batching multiple requests shorter than $L_m$ or individually scheduling requests longer than $L_m$. For decoding instances, we set $L_m$ as the largest batch size.

**Combat busrtiness.** Burstiness in workloads can cause a deluge of KV caches to transfer from prefill to decoding instances, risking memory overload on decoding instances. To circumvent this, DistServe employs a "pull" method for KV cache transmission rather than a "push" approach – decoding instances fetch KV cache from prefill instances *as needed*, using the GPU memory of prefill instances as a queuing buffer. This way, the prefill instance can continue handling other prefill jobs by simply retaining the KV Cache in the GPU memory after processing the prompt. Hence, each type of instance operates at its own pace without complex coordination.

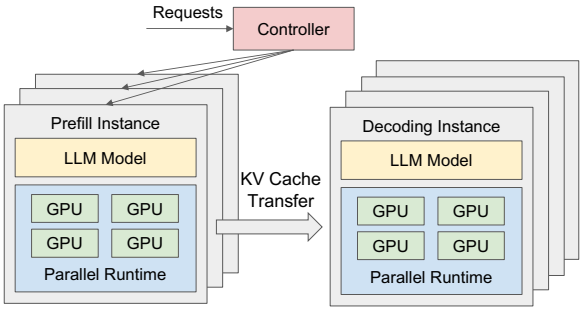



Figure 6: DistServe Runtime System Architecture

**Replaning.** The resource and parallelism plan in DistServe is optimized for a specific workload pattern, which may become suboptimal if the workload pattern changes over time. DistServe implement periodic replanning. A workload profiler monitors key parameters such as the average input and output length of the requests, the average arrival rate, etc. If a significant pattern shift is detected, DistServe will trigger a rerun of the placement algorithm based on recent historical data. This process is expedient – the proposed algorithm runs in seconds (§6.5) and reloading LLM weights can be completed within minutes – far shorter than the hourly scale at which real-world workload variations tend to occur.

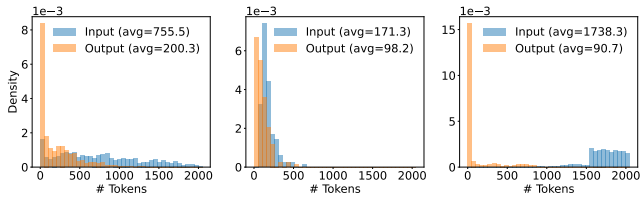
**Preemption and fault tolerance.** DistServe does not implement advanced runtime policies like preemption [26] and fault tolerance [58], which are complementary to disaggregation. Nevertheless, we discuss how they fit into DistServe. In DistServe, the FCFS policy can lead to a "convoy effect", where longer requests block shorter ones in the prefill stage. Incorporating preemptive strategies, as suggested in existing literature [53], could enhance efficiency and is feasible within our system's architecture. While not a primary focus in the current DistServe, fault tolerance is a critical aspect for consideration. In traditional colocation- and replication-based systems, a fault in one instance typically does not disrupt other replica instances. However, in DistServe, the dependency between prefill and decoding instances introduces the risk of fault propagation. For example, a fault in a single decoding instance mapped to multiple prefill instances could potentially cripple the entire service and cluster. We leave both as future work.

# 5 Implementation

DistServe is an end-to-end distributed serving system for LLMs with a placement algorithm module, a RESTful API frontend, an orchestration layer, and a parallel execution engine. The algorithm module, frontend, and orchestration layer are implemented with 6.5K lines of Python code. The parallel execution engine is implemented with 8.1K lines of

| Application | Model Size | TTFT | TPOT | Dataset |
|---|---|---|---|---|
| Chatbot OPT-13B | 26GB | 0.25s | 0.1s | ShareGPT [8] |
| Chatbot OPT-66B | 132GB | 2.5s | 0.15s | ShareGPT [8] |
| Chatbot OPT-175B | 350GB | 4.0s | 0.2s | ShareGPT [8] |
| Code Completion OPT-66B | 132GB | 0.125s | 0.2s | HumanEval [14] |
| Summarization OPT-66B | 132GB | 15s | 0.15s | LongBench [13] |

Table 1: Workloads in evaluation and latency requirements.




(a) ShareGPT (b) HumanEval (c) LongBench

Figure 7: The input and output length distributions of (a) ShareGPT, (b) HumanEval, and (c) LongBench datasets.

C++/CUDA code.

The placement algorithm module implements the algorithm and the simulator mentioned in §4 which gives the placement decision for a specific model and cluster setting. The frontend supports an OpenAI API-compatible interface where clients can specify the sampling parameters like maximum output length and temperature. The orchestration layer manages the prefill and decoding instances, responsible for request dispatching, KV cache transmission, and results delivery. It utilizes NCCL [6] for cross-node GPU communication and asynchronous CudaMemcpy for intra-node communication, which avoids blocking the GPU computation during transmission. Each instance is powered by a parallel execution engine, which uses Ray [35] actor to implement GPU workers that execute the LLM inference and manage the KV Cache in a distributed manner. It integrates many recent LLM optimizations like continuous batching [54], FlashAttention [20], PagedAttention [32] and supports popular open-source LLMs such as OPT [56] and LLaMA [51].

# 6 Evaluation

In this section, we evaluate DistServe under different sizes of LLMs ranging from 13B to 175B and various application datasets including chatbot, code-completion, and summarization. The evaluation shows that DistServe consistently outperforms the current state-of-the-art system across all the settings (§6.2). Specifically, DistServe can handle up to 7.4× higher rates and 12.6× more stringent SLO while meeting the latency requirements for over 90% requests. Additionally, we analyze the latency breakdown in DistServe to show the communication overhead is insubstantial thanks to our bandwidth-aware placement algorithm (§6.3) and do ablation studies of our techniques (§6.4). Finally, we profile the execution time of our placement algorithm (§6.5).

## 6.1 Experiments Setup

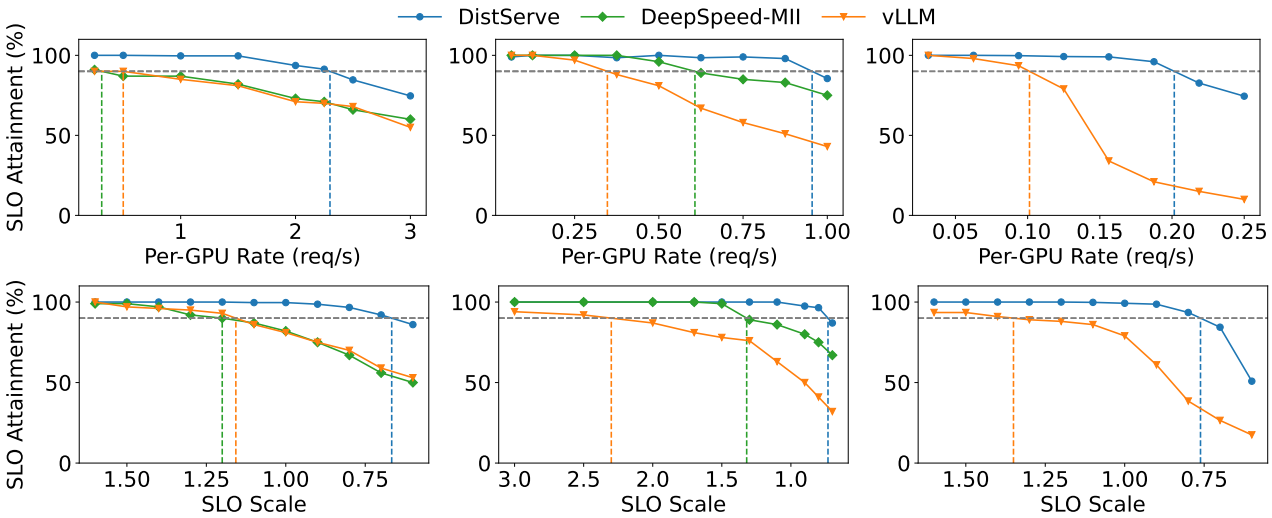
**Cluster testbed.** We deploy DistServe on a cluster with 4 nodes and 32 GPUs. Each node has 8 NVIDIA SXM A100-80GB GPUs connected with NVLINK. The cross-node bandwidth is 25Gbps. Due to the limited cross-node bandwidth, we use the low node-affinity placement algorithm (§2) for DistServe in most of the experiments except for the ablation study (§6.4) which uses simulation.

**Model and workloads setup.** Similar to prior work on LLM serving [32], we choose the OPT [56] model series, which is a representative LLM family widely used in academia and industry. Newer GPT model families are adopting memory-efficient attention mechanisms like GQA [10] and MQA [44]. DistServe will show better performance on these models because the transmission overhead is lower due to the decrease in KV cache size. We choose OPT which uses the classic MHA [52] to put enough pressure on the transmission overhead. We use FP16 precision in all experiments. For workloads, as shown in Table 1, We choose three typical LLM applications and set the SLOs empirically based on their service target because there exists no available SLO settings for these applications as far as we know. For each application, we select a suitable dataset and sample requests from it for evaluation. Since all the datasets do not include timestamps, we generate request arrival times using Poisson distribution with different request rates. Due to the space limit, we test the chatbot workload on all three OPT models and the other two workloads on OPT-66B, which matches the largest size in the recent open-source LLM series [51].

- **Chatbot** [1]: We use the ShareGPT dataset [8] for the chatbot application, which is a collection of user-shared conversations with ChatGPT. For OPT-13B, the TTFT SLO is set to 0.25s for responsiveness and the TPOT SLO is set to 0.1s which is higher than the normal human read speed. For OPT-66B and OPT-175B, we slightly relax the two SLOs due to the increase in model execution latency.
- **Code completion** [14]: We use the HumanEval [14] dataset for the code completion task. It includes 164 programming problems with a function signature or docstring which is used to evaluate the performance of code completion models. Since the code completion model is used as a personal real-time coding assistant, we set both SLOs to be stringent.
- **Summarization** [5]: It is a popular LLM task to generate a concise summary for a long article, essay, or even an academic paper. We use LongBench [13] dataset which contains the summarization task[4]. As shown in Figure 7, LongBench has much longer input lengths than the other two datasets. So we set a loose TTFT SLO but require a stringent TPOT.

**Metrics.** We use *SLO attainment* as the major evaluation metric. Under a specific SLO attainment goal (say, 90%), we are

[4]We capped the input lengths in LongBench because OPT's absolute positional embedding only supports a maximum length of 2048.




(a) OPT-13B (b) OPT-66B (C) OPT-175B

Figure 8: Chatbot application with OPT models on the ShareGPT dataset.

concerned with two things: the maximum per-GPU goodput and the minimal SLO the system can handle. We are particularly interested in an SLO attainment of 90% (indicated by the vertical lines in all curve plots), but will also vary the rate and latency requirements to observe how the SLO attainment changes. We also include the results in the Appendix for an SLO attainment of 99% to show the system performance under a more stringent SLO attainment target.

**Baselines.** We compare DistServe to two baseline systems:

- **vLLM [32]:** vLLM is a representative LLM serving system widely used in both academia and industry. It supports *continuous batching* [54] to increase throughput and *paged-attention* [32] to reduce memory fragmentation during KV cache allocation. However, it colocates the prefill and decoding computation to maximize the overall system throughput and struggles to meet the latency requirements cost-efficiently. Since vLLM only supports intra-op parallelism, we follow previous work [32] to set intra-op equals 1, 4, and 8 for the three OPT models, respectively.
- **DeepSpeed-MII [3]:** DeepSpeed Model Implementations for Inference (MII) supports *chunked-prefill* by decomposing long prompts into smaller chunks and composing with short prompts to exactly fill a target token budget. It mitigates but cannot eliminate the prefill-decoding interference caused by the long prefill job. We set its intra-op the same as vLLM for OPT-13B and OPT-66B for a fair comparison. However, DeepSpeed-MII cannot serve OPT-175B whose $vocab_size = 50272$ because its underlying kernel implementation requires $vocab_size/intra_op$ is a multiple of 8 where intra-op equals 8 does not satisfy. Setting intra-op equals 4 can satisfy this requirement but will cause the out-of-memory issue.

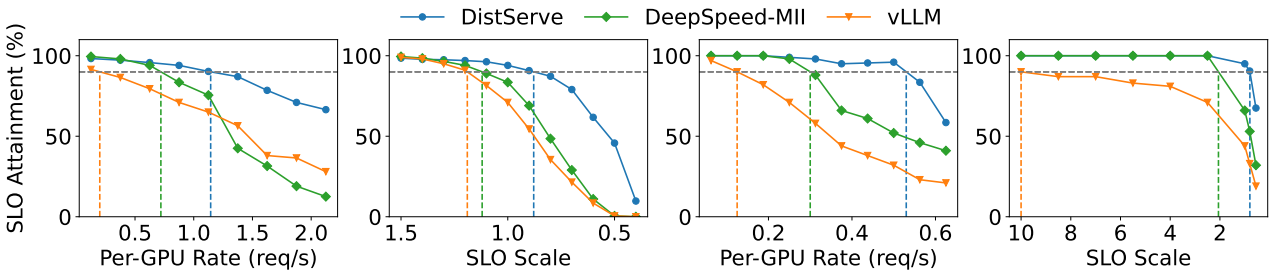
## 6.2 End-to-end Experiments

In this Section, we compare the end-to-end performance of DistServe against the baselines on real application datasets.

**Chatbot.** We evaluate the performance of DistServe on the chatbot application for all three OPT models. The first row of Figure 8 illustrates that when we gradually increase the rate, more requests will violate the latency requirements and the SLO attainment decreases. The vertical line shows the maximum per-GPU rate the system can handle to meet latency requirements for over 90% of the requests.

On the ShareGPT dataset, DistServe can sustain 2.0×–4.6× higher request rate compared to vLLM. This is because DistLLM eliminates the prefill-decoding interference through disaggregation. Two phases can optimize their own objectives by allocating different resources and employing tailored parallelism strategies. Specifically, by analyzing the chosen placement strategy[5] for 175B, we find the prefill instance has inter-op = 3, intra-op = 3; and the decoding instance has inter-op = 3, intra-op = 4. Under this placement, DistServe can effectively balance the load between the two instances on ShareGPT, meeting latency requirements at the lowest cost. This non-trivial placement strategy is challenging to manually find, proving the effectiveness of the algorithm. In the case of vLLM, collocating prefill and decoding greatly slows down the decoding phase, thereby significantly increasing TPOT. Due to the stringent TPOT requirements of chatbot applications, although vLLM meets the TTFT SLO for most requests, the overall SLO attainment is dragged down by a large number of requests that violate the TPOT SLO. Compared to DeepSpeed-MII, DistServe can sustain 1.6×–7.4× higher request rate. DeepSpeed-MII shows better performance

[5] All the placements chosen by DistServe can be found in Appendix B.




(a) Code Completion (b) Summarization

Figure 9: Code completion and summarization tasks with OPT-66B on HumanEval and LongBench datasets, respectively.

on larger models because the prefill job is larger and chunked-prefill mitigates the interference to some extent. However, due to the reasons discussed in §2.3, chunked prefill is slower than full prefill, so it struggles to meet the TTFT SLO as a sacrifice for better TPOT.

The second row of Figure 8 indicates the robustness to the changing latency requirements of the two systems. We fix the rate and then linearly scale the two latency requirements in Table 1 simultaneously using a parameter called *SLO Scale*. As SLO Scale decreases, the latency requirement is more stringent. We aim to observe the most stringent SLO Scale that the system can withstand while still achieving the attainment target. Figure 8 shows that DistServe can achieve 1.8×–3.2× more stringent SLO than vLLM and 1.7×–1.8× more stringent SLO than DeepSpeed-MII, thus providing more engaging service quality to the users.

**Code completion.** Figure 9(a) shows the performance of DistServe on the code completion task when serving OPT-66B. DistServe can sustain 5.7× higher request rate and 1.4× more stringent SLO than vLLM. Compared to DeepSpeed-MII, DistServe can sustain 1.6× higher request rate and 1.4× more stringent SLO. As a real-time coding assistant, the code completion task demands lower TTFT than chatbot, this leads to both systems ultimately being constrained by the TTFT requirement. However, in comparison, by eliminating the interference of the decoding jobs and automatically increasing intra-operation parallelism in prefill instances through the searching algorithm, DistServe reduces the average latency of the prefill jobs, thereby meeting the TTFT requirements of more requests.

**Summarization.** Figure 9(b) shows the performance of DistServe on the summarization task when serving OPT-66B. DistServe achieves 4.3× higher request rate and 12.6× more stringent SLO than vLLM. Compared to DeepSpeed-MII, DistServe achieves 1.8× higher request rate and 2.6× more stringent SLO. The requests sampled from LongBench dataset have long input lengths, which brings significant pressure to the prefill computation. However, due to the loose requirement of TTFT for the summarization task, the TPOT service quality becomes particularly important. Since vLLM collocates prefill and decoding phases, it experiences a greater slowdown in the decoding phase with long prefill jobs and fails to meet the TPOT requirement.

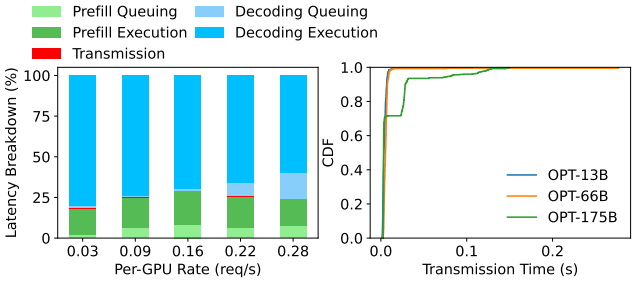



Figure 10: *Left:* Latency breakdown when serving OPT-175B on ShareGPT dataset with DistServe. *Right:* The CDF function of KV Cache transmission time for three OPT models.

The results above are all under the 90% SLO attainment target. We observe that DistServe can have better performance under a more stringent attainment target (say, 99%) and present the results in Appendix C.

## 6.3 Latency Breakdown

To understand DistServe's performance in detail, we make a latency breakdown of the requests in DistServe. We divide the processing lifecycle of a request in DistServe into five stages: *prefill queuing*, *prefill execution*, *transmission*, *decoding queuing*, and *decoding execution*. The total time consumed by all requests in each stage is then summed up to determine their respective proportions in the system's total execution time.

Figure 10(a) shows the latency breakdown for the OPT-175B models on the ShareGPT dataset. We chose OPT-175B because the KV Cache transmission is more demanding for larger models. In fact, even for OPT-175B, the KV Cache transmission only accounts for less than 0.1% of the total latency. Even by examining the CDF of the absolute transmission time shown in Figure 10(b), we observe that over 95% of requests experience a delay of less than 30ms, despite our testbed having only limited cross-node bandwidth. This is due to the algorithm described in §4.2, where we require the prefill and decoding instance to maintain the same stage on one machine, enabling the use of intra-node NVLINK bandwidth for transmission, thus significantly reducing transmission delay.

| Rate (req/s) | vLLM | | DistServe-Low | |
|---|---|---|---|---|
| | Real System | Simulator | Real System | Simulator |
| 1.0 | 97.0% | 96.8% | 100.0% | 100.0% |
| 1.5 | 65.5% | 65.1% | 100.0% | 100.0% |
| 2.0 | 52.8% | 51.0% | 99.3% | 99.3% |
| 2.5 | 44.9% | 46.1% | 87.3% | 88.3% |
| 3.0 | 36.7% | 38.3% | 83.0% | 84.1% |
| 3.5 | 27.8% | 28.0% | 77.3% | 77.0% |
| 4.0 | 23.6% | 24.1% | 70.0% | 68.9% |

Table 2: Comparison of the SLO attainment reported by the simulator and the real system under different rates.

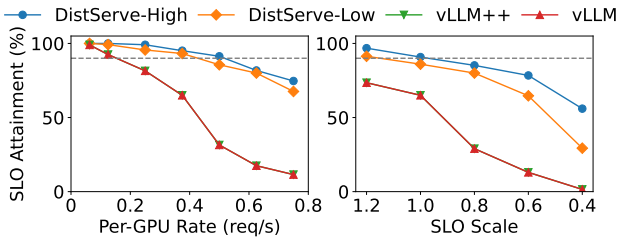



Figure 11: Ablation experiments.

## 6.4 Ablation Studies

We study the effectiveness of the two key innovations in DistServe: disaggregation and the placement searching algorithm. In §6.2, we choose the default parallelism setting for vLLM following its original paper [32]. So we implement "vLLM++" which enumerates different parallelism strategies and chooses the best. For DistServe, We also compare the placement found by Alg. 2 (DistServe-Low) with the one found by Alg. 1 (DistServe-High) which has fewer searching constraints and assumes high cross-node bandwidth. Since vLLM does not support inter-op parallelism and our physical testbed does not have high cross-node bandwidth, we use simulation for this experiment.

**Simulator accuracy.** Noticing that DNN model execution [24] has high predictability, even under parallel settings [33, 59]. We study the accuracy of the simulator in Tab. 2. For "vLLM" and "DistServe-Low", we compare the SLO attainment reported by the simulator and by real runs on our testbed under different rates. The error is less than 2% in all cases, verifying the accuracy of our simulator.

**Results.** Figure 11 shows the performance of the four systems when serving OPT-66B on the ShareGPT dataset. "vLLM++" has the same performance as "vLLM" because we find the default parallelism setting (intra-op=4) has the best per-GPU goodput. This further demonstrates the importance of disaggregation. The interference between the prefill and decoding phases significantly reduces the potential performance improvement through adjusting parallelism. In contrast, "DistLLM-High" can achieve further improvements over "DistLLM-Low" because it is not constrained by the deployment constraint that the prefill and decoding instance on one node should share the same model stage. Through disaggregation, we can use tailored parallelism strategies for prefill and decoding instances and optimize their targets without the coupling effects.

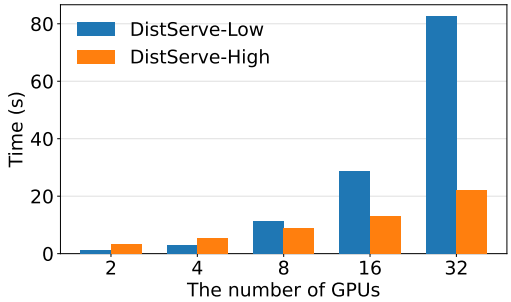



Figure 12: Algorithm Running Time

## 6.5 Algorithm Running Time

Figure 12 shows the running time for Alg. 1 (DistServe-Low) and Alg. 2 (DistServe-High) on an AWS m5d.metal instance with 96 cores as the number of GPUs ($N \times M$) provided to a single instance increases. According to the results, DistServe scales well with the number of GPUs and is independent of the model size. This is because the simulator only simulates discrete events and the running time is the same no matter how big the model is. On the other hand, both algorithms are highly parallelizable, as the searches for different parallelism strategies are independent of each other, allowing the execution time of the algorithms to accelerate almost linearly with more CPU cores.

As the number of GPUs increases, the execution time of "Dist-Low" becomes higher than that of "Dist-High". This is because the search for parallelism strategies for prefill and decoding instances in "Dist-High" is independent and can be parallelized. But for "Dist-Low", due to additional restrictions on deployment, we need to enumerate all the possible intra-node parallelism combinations for prefill and decoding instances. Even so, the execution time of the algorithm is in minutes, and since it only needs to be executed once before each redeployment, this overhead is acceptable.

# 7 Discussion

In this paper, we focus on the goodput-optimized setting and propose DistServe under the large-scale LLM serving scenario. As LLMs are widely used and deployed across various service scenarios with different optimization targets and resource limits, it becomes almost impossible to find a one-size-fits-all solution that effectively addresses all aspects of LLM serving. In this section, we discuss the pros and cons of DistServe and potentially better solutions in other scenarios.

**Throughput-optimized scenarios.** In offline applications that are not latency-sensitive, users typically have lower requirements for response time [45]. This allows serving systems to shift focus towards maximizing overall throughput instead of goodput and the effectiveness of DistServe may be

compromised. In this case, techniques such as chunked-prefill with piggyback [3, 9] may be preferred since it can fill each batch to the compute-bound threshold, thereby maintaining higher GPU utilization in every iteration.

**Resource-constrained scenarios.** Small-scale enterprises and individual researchers often lack the resources to deploy LLMs on large-scale clusters [45,48]. In resource-constrained scenarios, such as environments with only a few or even a single GPU, the design space for DistServe is significantly limited. It struggles or even fails to adjust the parallel strategies and resource allocation to effectively enhance serving performance. In this case, simpler architectural choices like non-disaggregated systems [3, 32] may reduce deployment complexity and optimize operational efficiency.

**Long-context scenarios.** Nowadays, more and more GPT models support extremely long contexts, such as Claude-3 [11], Gemini-1.5 [22], and Large World Model (LWM) [34], which all have a 1M context window. In such scenarios, the transmission overhead will increase as the size of the KV cache grows linearly with the prompt length. However, the prefill computation grows quadratically, so the relative duration of transmission and prefill job decreases. Meanwhile, a longer context further exacerbates the disparity in computational demands between prefill and decoding jobs, leading to increased interference between them. Therefore, the disaggregation approach proposed in DistServe remains promising in long-context serving.

## 8 Related Work

**Inference serving.** There has been plenty of work on inference serving recently. They range from general-purpose production-grade systems like TorchServe [7] and NVIDIA Triton [19] to systems optimized specifically for Transformer-based LLMs [9, 18, 21, 33, 50, 53, 54, 60]. Among them, Orca [54] introduces continuous batching to increase throughput. vLLM [32] proposes paged-attention for fine-grained KV cache management. SARATHI [9] suggests a chunked-prefill approach, splitting a prefill request into chunks and piggybacking decoding requests to improve hardware utilization. FastServe [53] implements iteration-level preemptive scheduling to mitigate the queuing delay caused by long jobs. However, they all employ a colocation approach for prefill and decoding processing, thus leading to severe interference. There are also concurrent works such as Splitwise [38], TetriInfer [27] and DéjàVu [49] which adopt similar disaggregation idea to optimize LLM inference, further confirming the effectiveness of this method. Differently, DistServe emphasizes the goodput optimization scenario more and takes a closer look at the aspect of network bandwidth.

**Goodput-optimized systems.** Optimizing goodput is a hot topic in DL applications. Pollux [39] improves scheduling performance in DL clusters by dynamically adjusting resources for jobs to increase cluster-wide goodput. Sia [29] introduces a heterogeneous-aware scheduling approach that can efficiently match cluster resources to elastic resource-adaptive jobs. Clockwork [23] and Shepherd [55] provide latency-aware scheduling and preemption to improve the serving goodput, but they only target traditional small models. AlpaServe [33] focuses on LLMs, employing model parallelism to statistically multiplex the GPU execution thus improving the resource utilization. However, it only targets the non-autoregressive generation. DistServe is the first work to optimize the goodput for autoregressive LLM inference.

**Resource disaggregation.** Resource disaggregated systems [17, 25, 43] decouple the hardware resources from the traditional monolithic server infrastructure and separate them into resource pools to manage independently. It allows for more flexible, efficient, and scalable deployment and increases resource utilization. Many applications benefit from a truly disaggregated data center with high-speed network bandwidth and heterogenous hardware support [12, 30, 57]. DistServe shares the concept by disaggregating its system components, allowing for independent resource scaling and management.

**Model parallelism for training.** DistServe is orthogonal to the large body of work on model parallelism in training [28, 36, 40, 46, 59]. As described in §3.3, inference-serving workloads have unique characteristics not found in training settings. Where these systems do intersect with DistServe, is in their methods for implementing model parallelism along various dimensions. DistServe can integrate new parallelism optimizations into its placement searching algorithm.

## 9 Conclusion

We present DistServe, a new LLM serving architecture that disaggregates the prefill and decoding computation. DistServe maximizes the per-gpu goodput – the maximum request rate that can be served adhering to the SLO attainment goal for each GPU provisioned, hence resulting in up to 7.4× lower cost per LLM query with guaranteed satisfaction of SLOs. Our findings affirm that as latency becomes an increasingly important metric for LLM services, prefill and decoding disaggregation is a vital strategy in promising improved performance and service quality guarantees.

**Acknowledgments.** We sincerely thank our shepherd and the anonymous reviewers for their valuable feedback. This work was supported by the National Natural Science Foundation of China under the grant numbers 62172008, 62325201, and the National Natural Science Fund for the Excellent Young Scientists Fund Program (Overseas). Junda Chen is supported by UCSD fellowship and Hao Zhang is supported by UCSD faculty startup fund. Xin Jin is the corresponding author. Yinmin Zhong, Xuanzhe Liu, and Xin Jin are also with the Key Laboratory of High Confidence Software Technologies (Peking University), Ministry of Education.